BATTLE ARENA · TEAM PROJECT

Battle Arena

(게임 시스템 설계와 구현)

멀티플레이 통합 서버 · 점프패드 & 맵 기믹 · EQS 기반 추격 AI · GameFlow

양지성 (GitHub: GimBOBeat123)

Unreal Engine 5.6 · C++ & Blueprint



목차

01-02	프로젝트 개요 · 내가 맡은 범위	C++ / Blueprint
03-05	멀티플레이 서버 인프라	로비 · 매치 프로세스 분리
06-10	점프패드 · 맵 기믹	비대칭 탄도 계산
11-15	추격 AI 컨트롤러	BP_QuakeAIController2
16	EQS 쿼리 설계	전투 · 후퇴 포지션
17-19	GameFlow · Inventory	화면 흐름 · 무기 스왑
20-23	트러블슈팅 · 검증	실서버 테스트 · 영상
24	AI 도구(Claude Code) 활용	사용 범위 · 경계
25	시네마틱 트레일러	YouTube

01 프로젝트 개요

팀 인원

기획 4명 · 프로그래밍 5명 (총 9명)

개발 기간

2025.12 ~ 진행 중

장르 / 플랫폼

아레나 FPS · PC

모드

캠페인(협동 vs AI) · 매치(PvP)

엔진

Unreal Engine 5.6

레포지토리

github.com/Qusbell/ShootingArena

멀티플레이

로비 서버 · 매치 서버 분리 구조

맵 기믹

점프패드 비대칭 탄도 계산 커스텀 노드

게임 AI

EQS 전투 · 후퇴 포지션 쿼리 + 추격 컨트롤러

02 내가 맡은 범위

멀티플레이 서버

로비 / 매치 서버 분리, 재접속 · 닉네임 · 세션 구분 C++ 구현

점프패드 · 맵 기믹

비대칭 중력 탄도 커스텀 노드 + 전용 컴포넌트, 포탈 · 이동발판

추격 AI 컨트롤러

BP_QuakeAIController2 — 지각 · 기억 · 무기별 교전거리 산출을 Blackboard 로 전달

EQS 쿼리 설계

전투 포지션 · 후퇴 위치를 계산하는 쿼리와 컨텍스트 전부 제작

GameFlow UI

메인메뉴 → 로비 → 맵 / 난이도 선택 → 매치 → 결과 흐름

Inventory

무기 스왑 로직, 인벤토리 인터페이스, HUD 연동 일부

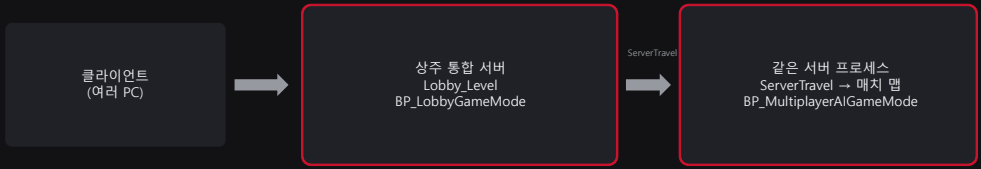
C++ — 멀티플레이 서버 · 점프패드 / Blueprint — 추격 AI · EQS · GameFlow · Inventory

03 멀티플레이 — 통합 서버 구조

설계 결정	구현
매치를 별도 프로세스로 스폰하지 않고, 하나의 상주 서버가 매치 맵으로 전환	ALobbyGameMode::TravelToMatch — World->ServerTravel(URL, bAbsolute) 로 서버·클라 일괄 이동
매치 → 로비 복귀는 접속한 인간 플레이어 전원 투표	AMyPlayerState 투표 + AMyReconnectionGameMode::CheckReturnToLobbyVotes → ServerTravel(Lobby)
슬롯·방장은 복제, 매치를 왕복해도 상태 유지	ALobbyGameState::Slots(DOREPLIFETIME) + GameInstance 에 AI 슬롯·방장 주소·닉네임 저장
방장 권한은 서버 RPC 진입점에서 게이트	ALobbyPlayerController 의 Server_* RPC 전부 IsHost() 선검사

캠페인은 별개 — 진입 시 로컬 전용 서버 프로세스를 새로 띄우고 종료 시 닫음 (ULocalDedicatedServerLibrary)

04 멀티플레이 — 상주 서버 흐름



- 학교 서버 PC 에 ShootingArenaServer.exe 를 상주 실행(RunServer.bat, port 7777) → 다른 PC 클라이언트가 IP:7777 로 접속
- 방장이 게임 시작 → TravelToMatch 가 서버 자신을 매치 맵으로 ServerTravel, 접속된 모든 클라가 함께 이동 (클라 open 불필요)
- 매치 중 결과창에서 인간 플레이어 전원이 '로비로' 투표하면 서버가 ServerTravel(Lobby_Level) 로 복귀
- 닉네임 · AI 슬롯 · 방장은 GameInstance 에 저장 → 매치 왕복 · 맵 변경 후에도 유지

05 멀티플레이 서버 — 실행 화면



상주 서버 콘솔 로그 + 로비 UI



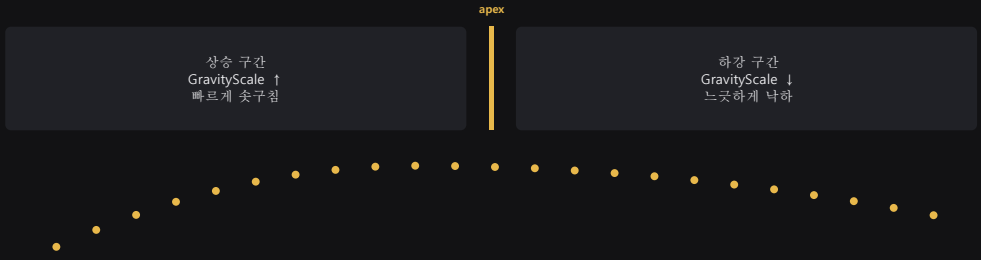
캠페인 진입 시 로컬 전용 서버 준비 화면

06 점프패드 탄도 계산

설계 결정	구현
발사각 고정 + 비대칭 중력(상승 / 하강 GravityScale 분리) 궤적	SuggestJumpPadVelocity — 상승구간 닫힌 해 / 하강구간 이분탐색 분기
타이머 없이 Z축 속도 부호로 중력 전환	UJumpPadFlightComponent::TickComponent — 매 틱 Velocity.Z 부호 판정
계산은 BP 커스텀 노드, 상태 전환은 Actor Component	UCustomWrapperLibrary / UJumpPadFlightComponent 로 책임 분리

검증: 에디터에서 여러 목표 지점에 대해 실제 착지 정확도 확인

07 점프패드 — 비대칭 포물선



상승 중 명중

중력이 GUp 하나뿐 → 닫힌 이차식으로 즉시 계산

하강 중 명중

GUp · GDown 혼합 → 오차 함수를 이분 탐색 60회로 수치해

08 점프패드 블루프린트 — 커스텀 노드를 쓰는 로직

공통 흐름 (BPA_YJS_ApexTime / LaunchAngle_JumpPad)

밟은 캐릭터 저장 → 컨트롤러 캐스팅으로 AI 여부 판별 → 연결된 타깃 액터 위치 + 시작 위치로 커스텀 노드 호출 → LaunchCharacter 로 발사.

ApexTime 패드

SuggestJumpPadVelocityByApexTime(ApexTime, GravityZOverride) 로 속도 계산 + K2_ProjectPointToNavigation 으로 착지점을 NavMesh 에 투영해 AI 도달 가능성 확인.

LaunchAngle 패드 + C++ 컴포넌트 연동

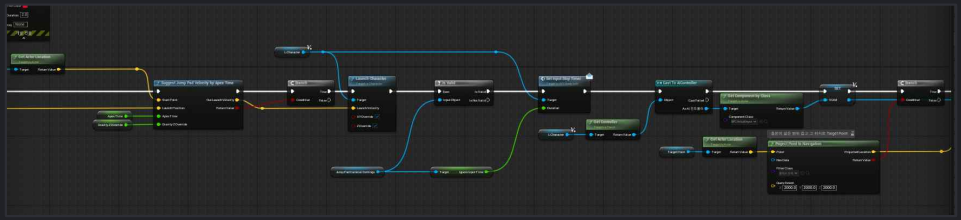
SuggestJumpPadVelocity(Rise 3.0 / Fall 0.8 / Angle 60°) → JumpPadFlightComponent 가 붙어 있으면 BeginFlight(Rise, Fall) 호출로 비대칭 중력 궤적 활성화.

AI 지원

밟은 게 봇이면 VirtualInputAIMovement 컴포넌트를 찾아 SetInputStopTimer(CommonSettings.IgnoreInputTime) — 발사 직후 봇 입력이 궤적을 흐트러뜨리지 않도록 잠금.

09 점프패드 BP — ApexTime 패드

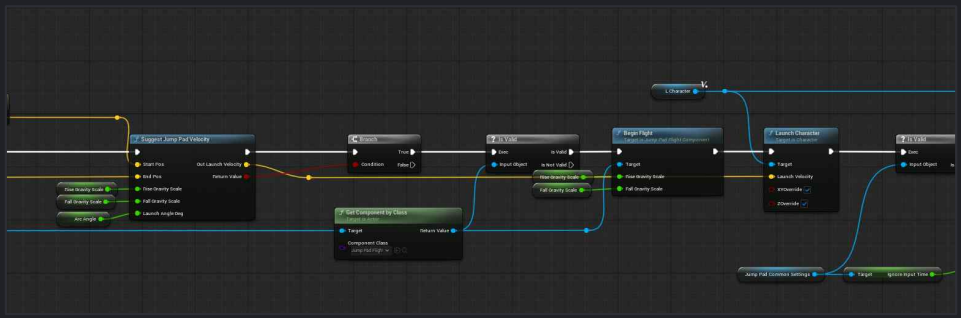
밟은 캐릭터 → 컨트롤러 캐스팅으로 AI 여부 판별 → SuggestJumpPadVelocityByApexTime(ApexTime, GravityZOverride) 로 속도 계산 → K2_ProjectPointToNavigation 으로 착지점을 NavMesh 에 투영해 AI 도달 가능성 확인 → LaunchCharacter 로 발사



BPA_YJS_ApexTime_JumpPad 이벤트 그래프

10 점프패드 BP — LaunchAngle 패드

SuggestJumpPadVelocity(Rise 3.0 / Fall 0.8 / Angle 60°) 로 발사 벡터 계산 → JumpPadFlightComponent 가 붙어 있으면 BeginFlight(Rise, Fall) 호출로 비대칭 중력 궤적 활성화 → LaunchCharacter 로 발사



BPA_YJS_LaunchAngle_JumpPad 이벤트 그래프

11 커스텀 노드 · 클래스 구조

UCustomWrapperLibrary : UBlueprintFunctionLibrary

cpp 전용 기능을 BP 노드로 노출. static UFUNCTION + meta(WorldContext, Keywords). 반환은 전용 enum(ENavPathTestResult 등). 노드: SuggestJumpPadVelocity · TestPathExistsSync · GetActiveBehaviorTreePath …

UJumpPadFlightComponent : UActorComponent

BeginPlay 가 LandedDelegate 바인딩 → BeginFlight 가 원래 GravityScale 저장 후 틱 on → TickComponent 가 Velocity.Z 부호로 Rise / Fall 전환 → 착지 시 원복.

UJumpPadCommonSettings : UDataAsset

모든 패드가 공유하는 값(IgnoreInputTime)만 보관. 패드 고유값(목표 지점 · ApexTime)은 액터에 유지 — 값을 공유/고유로 분리.

12 추격 AI 컨트롤러 (BP_QuakeAIController2)

역할 분담

BT_QuakeTree 는 '판단'만 담당. 컨트롤러가 지각 · 기억 · 교전거리 계산을 처리해 BB_QuakeBoard 키로 넘긴다. (BT 트리 내부와 무기 가중치 테이블은 팀원 작업)

초기화 (ReceivePossess)

UseBlackboard(BB_QuakeBoard) → 타깃 구조체 초기화 → BPC_RespawnHelper.BindRespawn(5s) → OnTakeAnyDamage · OnSwapItem 바인딩 → RunBehaviorTree.

타깃 추적 (AIPerception)

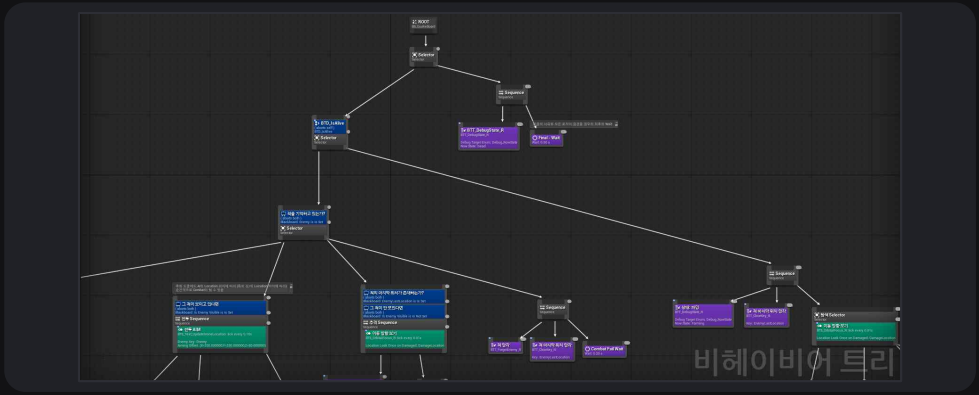
살아있는 슈터 감지 → ST_TargetInfo{Target, IsVisible, VisibleSince} 갱신. 시야 상실 시 ProjectPointToNavigation 으로 마지막 위치를 NavMesh 에 투영 → BB EnemyLastLocation + 3.초 상실 타이머.

무기별 교전 거리

OnSwapItem → BPC_AIDataCache.GetWeaponChangeRow → 사거리 커브를 내 C++ 노드 GetPeakKeyFromRuntimeFloatCurve 로 넘겨 최적 교전거리 산출 → BB Min / Max 교전거리 ±100.

13 추격 AI — 𝔹 최상위 구조

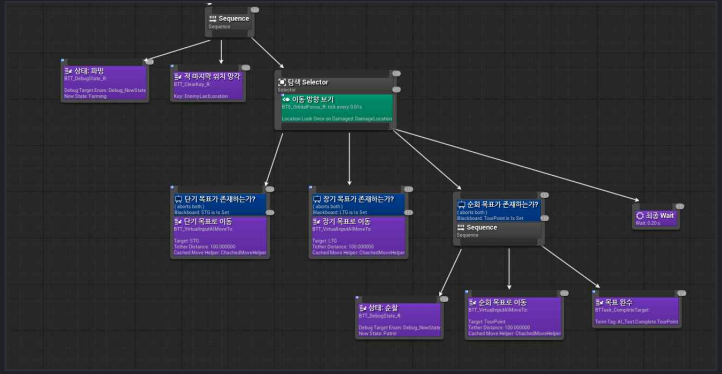
BT_QuakeTree 최상위 — 생존 확인 → 적 인지 여부 → 전투 / 추적 분기. 세부 조건 노드는 팀 협업 자산이라, 컨트롤러가 채운 Blackboard 값을 BT 가 소비하는 접점만 다룸



BT_QuakeTree 최상위 구조

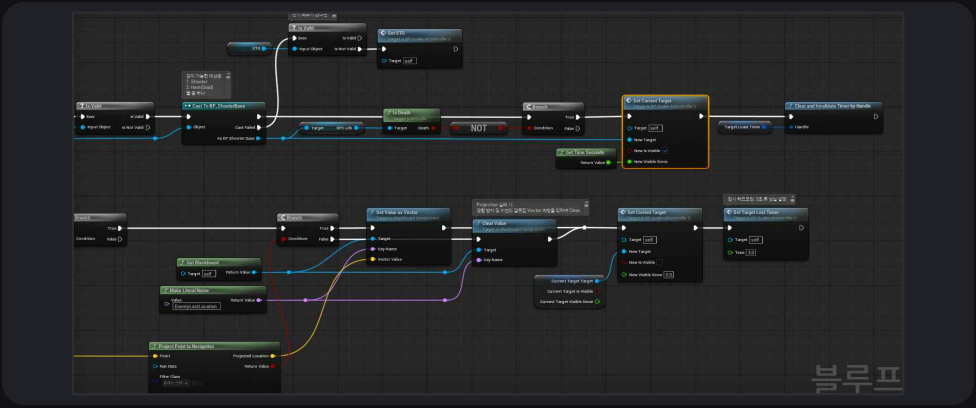
14 추격 AI — BT 탐색 Selector

컨트롤러가 채운 STG(단기 목표) / LTG(장기 목표) / TourPoint(순찰 지점)를 BT 가 그대로 소비해 이동 타깃을 고르는 지점



탐색 Selector — STG / LTG / TourPoint

15 추격 AI — 퍼셉션



OnTargetPerceptionUpdated — Set Current Target

16 EQS — 전투 · 후퇴 포지션 쿼리

EQS_CombatPosition — 전투 포지션 쿼리

Donut 제너레이터(반경 700~900, 적 방향 호) → Pathfinding·시야(Trace) 필터 → 적→쿼리어 방향 내적(Dot2D)으로 측면 위치를 필터·스코어.

EQS_RetreatPosition — 후퇴 위치 쿼리

SimpleGrid(2000 / 200) → 적과 거리 800~1300 필터 → 거리 스코어(800~2000 클램프) + 시야 없음(엄폐) 스코어. 옵션 2개로 fallback 구성.

EQC_PlayerForwardAxis — 컨텍스트 BP

ProvideLocationsSet 오버라이드. 블랙보드에서 타깃을 읽어 캐스팅 → 플레이어 전방축 ±700 / 800 / 900 지점 6개를 컨텍스트 포인트로 생성.

17 GameFlow · Inventory

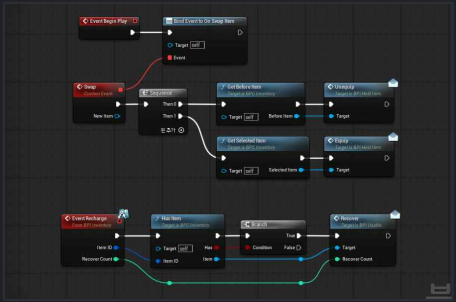
GameFlow — 화면 흐름 (Widget)

WBP_Lobby(슬롯 리스트 + 방장 라벨, MainAction 버튼 = Server_StartGame / Server_SetReady 토글) → WBP_MapSelect(캠페인 맵·캐릭터 선택 → GameInstance 저장) → WBP_Result(캠페인 / 매치 분기, 로비 재접속).

Inventory (BPC_Inventory) — 무기 스왑

BPI_Inventory 인터페이스 이벤트(Pickup / SwapNext / SwapPrev / Recharge) 구현. All_Items 배열 + OnSwapItem 델리게이트, 아이템에 Equip / Unequip 메시지 전달.

18 Inventory — 실행 화면



Swap → Equip / Unequip



Pickup (권한 게이트)

19 GameFlow — 레벨 선택



레벨 선택 화면 — 맵 썸네일 + 캐릭터 선택

20 트러블슈팅

① 결과창 '로비로' 누르면 서버 다운 · 클라 전원 튕김

매치가 별도 프로세스라 매치 종료 시 프로세스가 죽으며 접속 클라가 전부 끊김. 매치를 상주 서버의 ServerTravel 로 전환해, 서버가 살아있는 채 로비 맵으로 복귀하도록 재설계.

② 매치를 왕복하면 AI 슬롯 · 방장 · 닉네임이 리셋됨

상주 서버가 계속 살아있어도 ServerTravel 로 GameState 는 새로 생성됨. AI 슬롯 구성 · 방장 주소 · 닉네임을 GameInstance 에 저장해두고, 로비가 (재)구성될 때 복원.

③ 비대칭 궤적, 공식 하나로 안 풀림

닫힌 해가 나오는 영역과 안 나오는 영역을 수학적으로 먼저 분리하고, 필요한 구간(하강 명중)만 이분 탐색으로 수치해.

21 검증 — 실서버 테스트

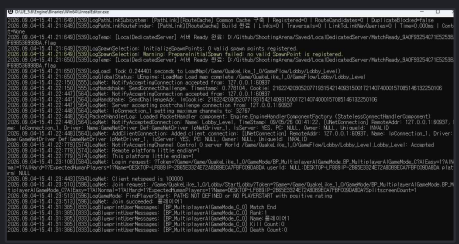
1. 학교 서버 PC 에 상주 데디케이티드 서버(ShootingArenaServer.exe)를 띄우고, 다른 PC 클라이언트가 IP 로 원격 접속하는 조건으로 반복 테스트

2. 리슨 서버에선 안 보이던 버그(닉네임 소실 · AI 슬롯 누적 · 결과창 복귀 시 클라 튕김)를 이 조건에서 발견 · 수정

3. 커밋을 시스템 단위로 분리해 회귀가 생겼을 때 원인 지점을 좁힘

✔ 원격 데디케이티드 서버 테스트 — 로비 입장 ~ 매치 ~ 로비 복귀 정상 확인

22 검증 — 실행 증거

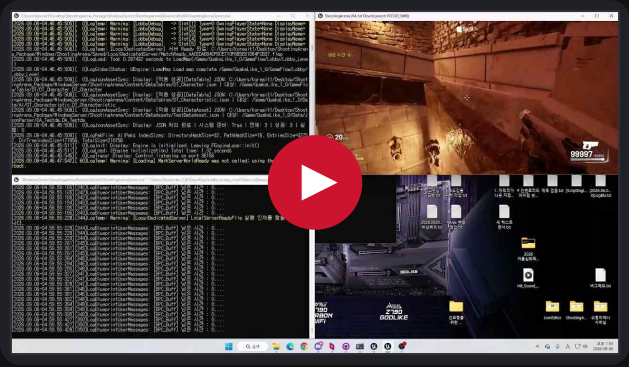


서버 매치 로그 (토큰 · Match End)



결과 화면 (닉네임이 다른 2인 세션)

23 검증 — 멀티플레이 서버 테스트 영상



▶ 멀티플레이 서버 테스트 영상 보기

https://youtu.be/VaP-1kpsadQ

24 AI 도구(Claude Code) 활용

AI로 가속한 부분	직접 한 부분
멀티플레이 C++ 골격 — 프로세스 스폰, 복제 보일러플레이트, RPC 구현	그 위의 아키텍처 — 로비 / 매치 분리 결정, 복제 대상 선정, 권한 경계 설계
점프패드 비대칭 포물선 방정식 유도 (닫힌 해 + 수치해)	점프패드 Blueprint 로직 전체 · 노드 / 컴포넌트 책임 분리 설계
생성된 C++ 초안	리뷰 · 수정 · 디버깅 · 실서버 검증 · 커밋 단위 분리

Blueprint 로직(추격 AI · EQS · GameFlow · Inventory)은 AI를 쓰지 않았으며, 생성 코드를 그대로 커밋한 적이 없습니다

25 시네마틱 트레일러



▶ 팀 제작 시네마틱 트레일러 보기

https://youtu.be/gmcgjWhXbic

정리

- 멀티플레이 서버 인프라를 로비 / 매치 프로세스 분리 구조로 설계하고 C++로 구현
- 점프패드는 비대칭 중력 궤적을 닫힌 해 / 수치해로 나눠 풀고 계산 · 상태 전환 책임을 분리
- 추격 AI · EQS · GameFlow · Inventory를 Blueprint로 직접 구현하고 실서버로 검증